

Reflection: three risks of shipping this to real users

1. The LLM misreads a hard constraint, or an explanation states something false

What goes wrong. The parser reads “₹600 per person” as ₹600 total, or “must have a projector” as a preference. The user gets confidently wrong results that look right. Separately, a rephrased explanation could claim “has a projector” for a room without one. That is the fastest way to lose a booker’s trust, and it can create a refund dispute.

Mitigation, partly built here:

- The LLM never touches listing data. Results, prices and amenities come from the dataset only; a test asserts every returned listing is byte-identical to its dataset record.
- Explanations are built from a per-listing fact sheet. LLM rephrasing is accepted only if every number, amenity and noise claim appears in those facts, otherwise the template is used. Tests cover both paths.
- Hard constraints the LLM adds must be backed by the user’s words (`reconcile.py`). An invented date, an unstated per-day or per-person budget, or an inferred space type is relaxed and shown as an assumption, never silently applied.
- The “What I understood” panel shows exactly how the request was read, including assumptions (“Budget read as total per hour, since no other unit was stated”). A misparse is visible before the user books.
- Next: make the chips editable, run the evaluation set in CI on every prompt or model change, and review logged queries where users immediately rephrase, which is a cheap misparse signal.

2. Ranking drifts toward expensive or already-popular listings

What goes wrong. Trust and rating signals reward listings that already have many reviews. New or cheaper hosts get little exposure, never collect reviews, and churn off the marketplace. Separately, if “premium” amenities correlate with price, soft-preference fit quietly pushes the top results upmarket, so users see a skewed view of what their budget can buy.

Mitigation:

- Bayesian rating uses a prior at the market mean, so a new listing starts at average, not zero. It isn’t buried, it just isn’t promoted.
- An explicit value term rewards headroom under the user’s budget. The weight is small but not zero.
- No paid boosting inside the score. Sponsored placement, if ever added, must be a separate labelled row.
- Next: monitor the median price of the top-3 against the median of all eligible listings per query, which is a direct measure of upmarket drift. Reserve an exploration slot for new listings. Retrain weights from booking outcomes rather than hand-tuning.

3. Availability is stale

What goes wrong. The prototype uses a weekly pattern, and even a live system caches. The user sees “Free Tue 13:00–15:00”, tries to book, and the slot is gone. The explanation was honest about the data but wrong about reality, and from the user’s side that is indistinguishable from hallucination.

Mitigation:

- Query the booking system for availability at search time, only for the candidate set, with a short cache TTL.
- Re-check and place a short hold when the user clicks “Book”. If the slot is gone, offer the next free slot at the same listing first; the matcher already computes “free 16:00–21:00 instead”.
- Show freshness (“availability checked 2 min ago”) and prefer instant-book listings for near-term requests, where staleness hurts most. Conversion already rewards instant-book.
- Measure the booking-failure rate caused by conflicts, and alert when it rises.